Hognose scanner notes

Keywords

```
bool
else
false
float
if
int
str
true
while
```

Characters in Σ (shown once from top to bottom)

```
[a-zA-Z_][a-zA-Z0-9_]*  : identifiers
[.] : float literals
[()] : functions, paren-expressions
[{}] : block-expressions
["]  : str literals
[#]  : comment
[<>=!&|+-*/%] : operators
[\]  : line continuation
\s   : any whitespace character
```

Implementation Notes: All whitespace characters constitute separators between tokens, but only participate in `whitespace` and `eof` tokens for position keeping. Line continuation can be treated as a special case in the scanner, similar to whitespace (see “Newline continuation” below).

Token Classes

```
brace-close
brace-open
comment
eof
identifier
keyword
line-continuation
literal-bool
literal-float
literal-int
literal-str
operator
paren-close
paren-open
whitespace
```

REs corresponding to token classes

The lexeme is the portion of the input stream matching the RE. RE's below are in Python `re` syntax, assuming single quotes surround them, e.g. `r'<regex>'`.

```
Token class   : RE
--------------:--------------------------------------------------
brace-close   : }
brace-open    : {
comment       : #.*\n
eof           : \s*<EOF>    (<EOF> is EOF state, not a character)
identifier    : [a-zA-Z_][a-zA-Z0-9_]*
keyword       : bool|else|false|float|if|int|str|true|while
line-continuation : \\[ \t]*\n
literal-bool  : false|true
literal-float : [0-9]+\.[0-9]+
literal-int   : [0-9]+
literal-str   : "[^"\n]*"
operator      : <=|>=|==|!=|&&|\|\||[<>=!+\-*/%]
paren-close   : \)
paren-open    : \(
whitespace    : \s+
```

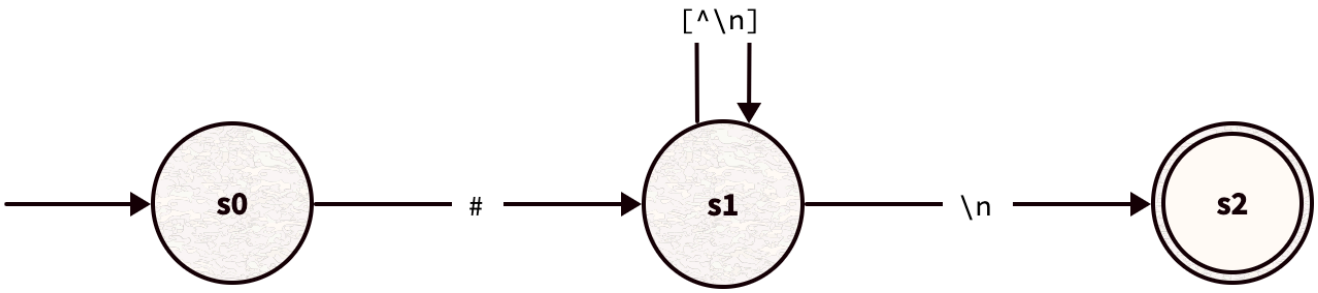
DFAs

The following DFAs are trivial (single transition):

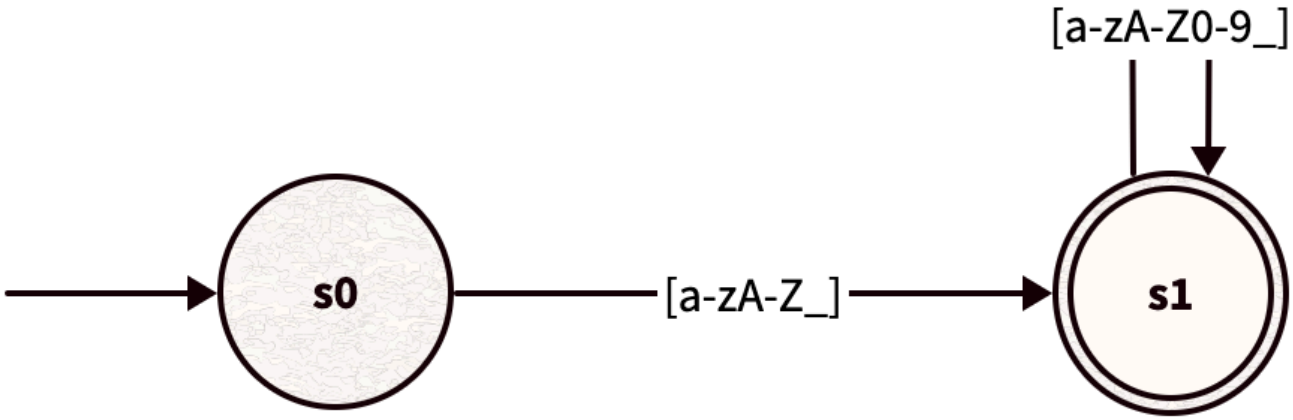
```
`brace-close`, `brace-open`, `paren-close`, `paren-open`
```

The following DFAs are non-trivial:

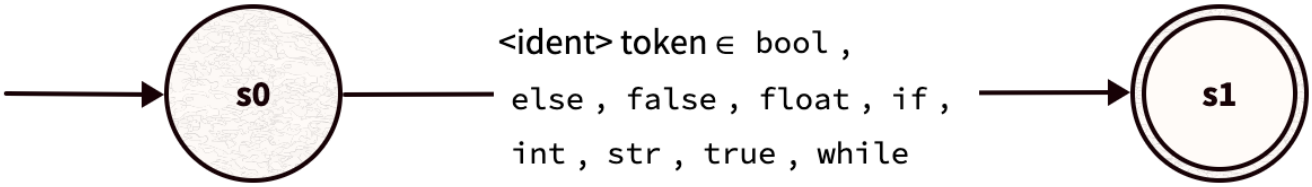
- `comment` :



- `identifier` :



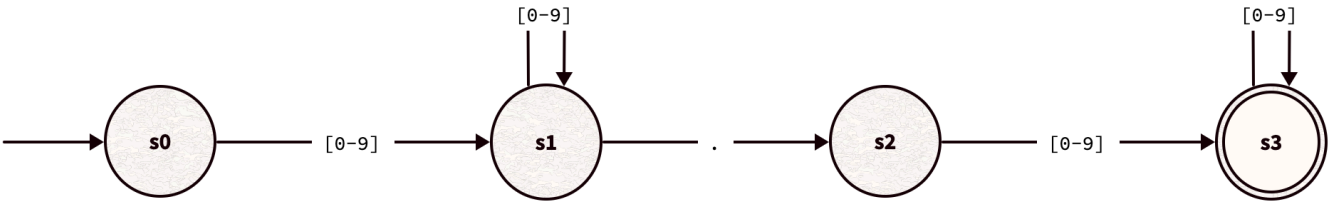
- `keyword` :



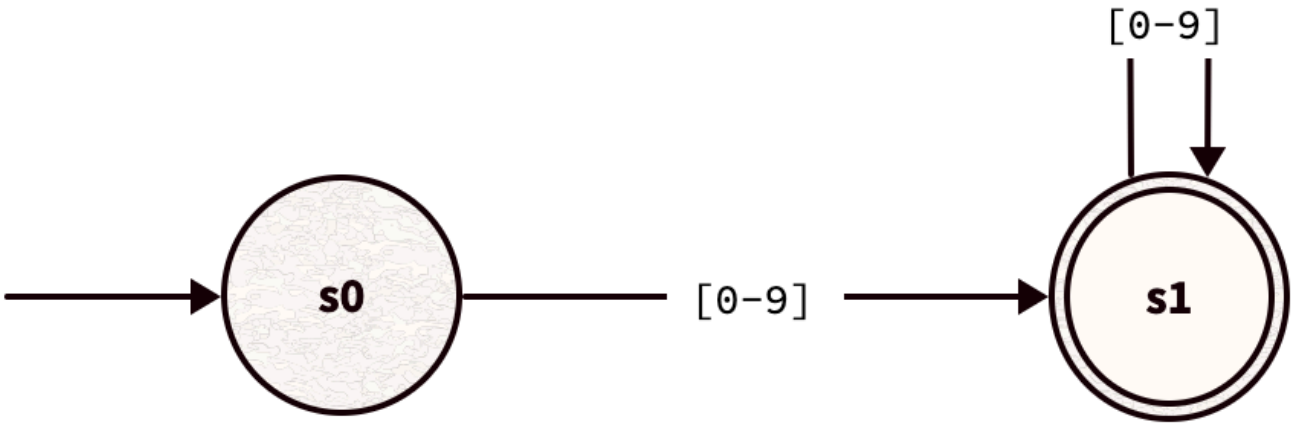
- `literal-bool` :



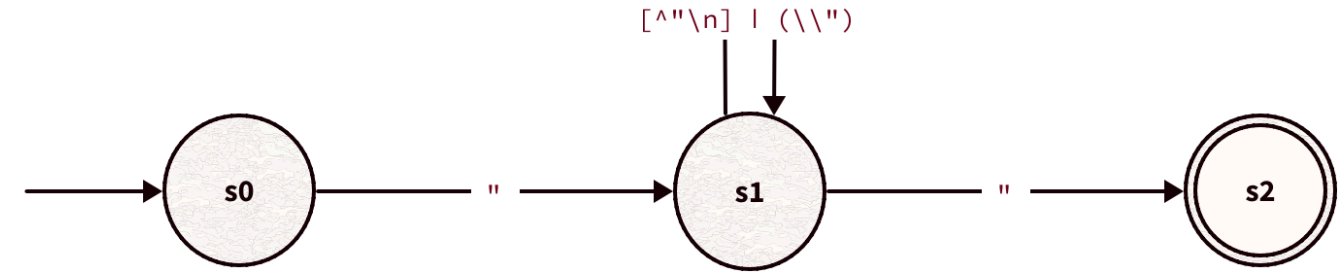
- `literal-float` :



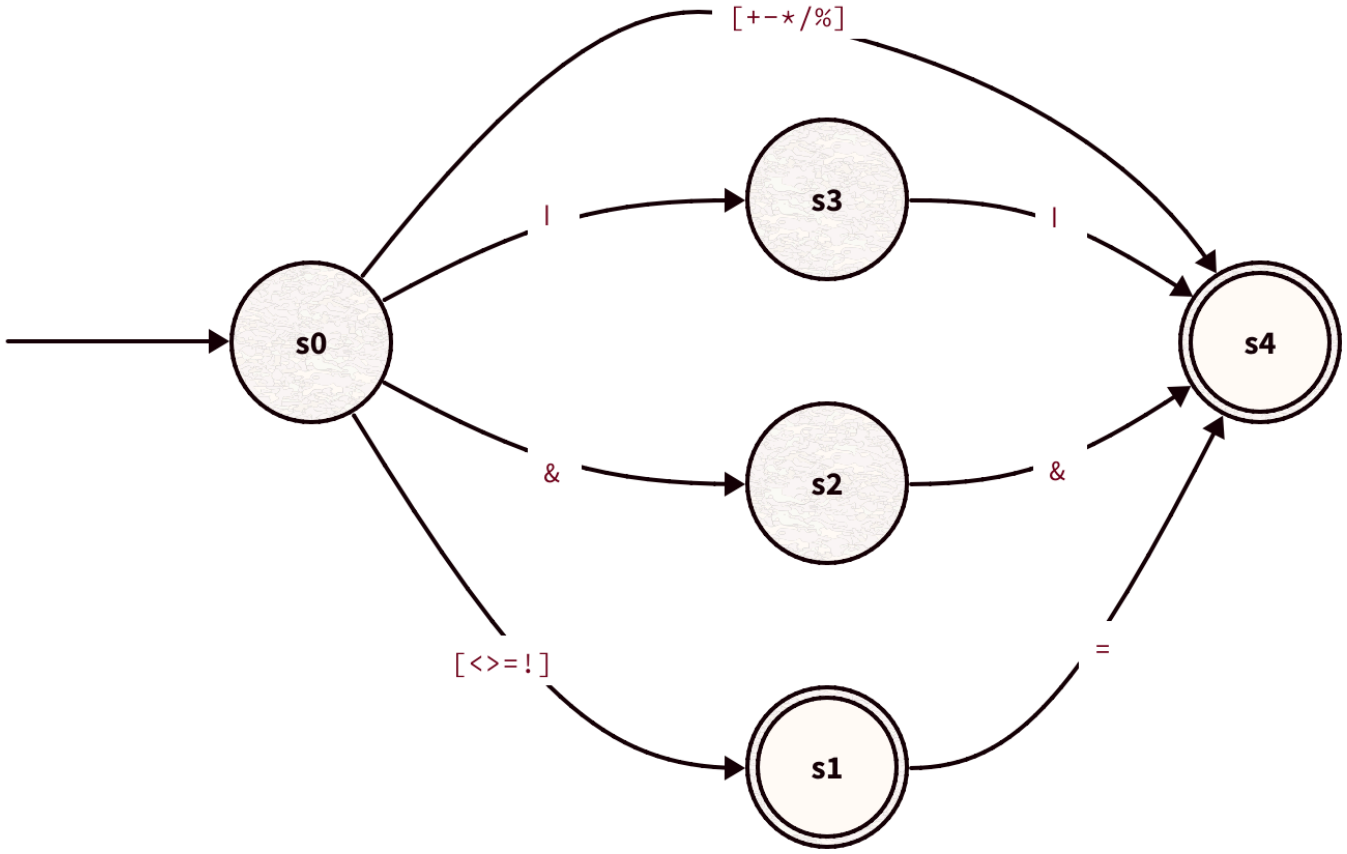
- `literal-int` :



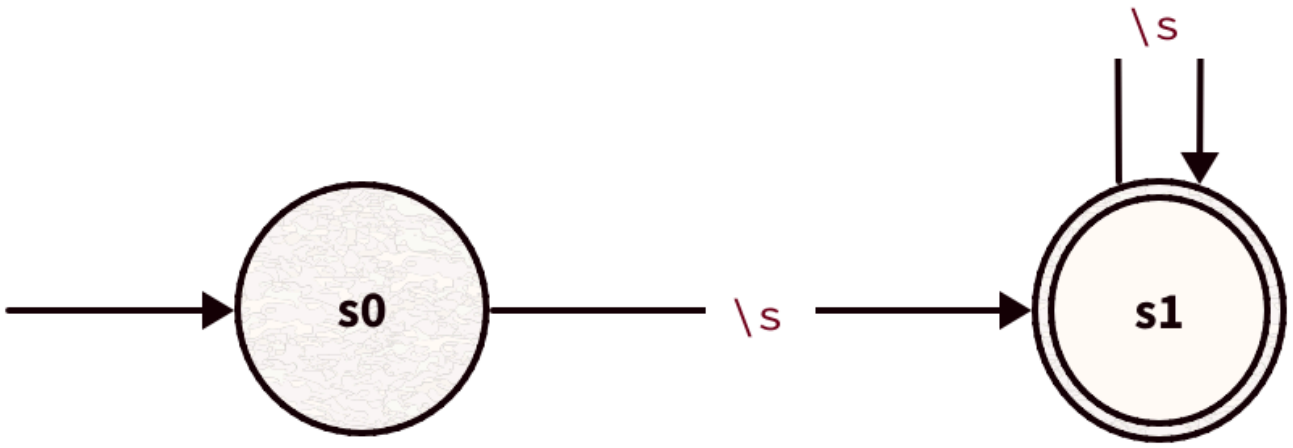
- `literal-str` :



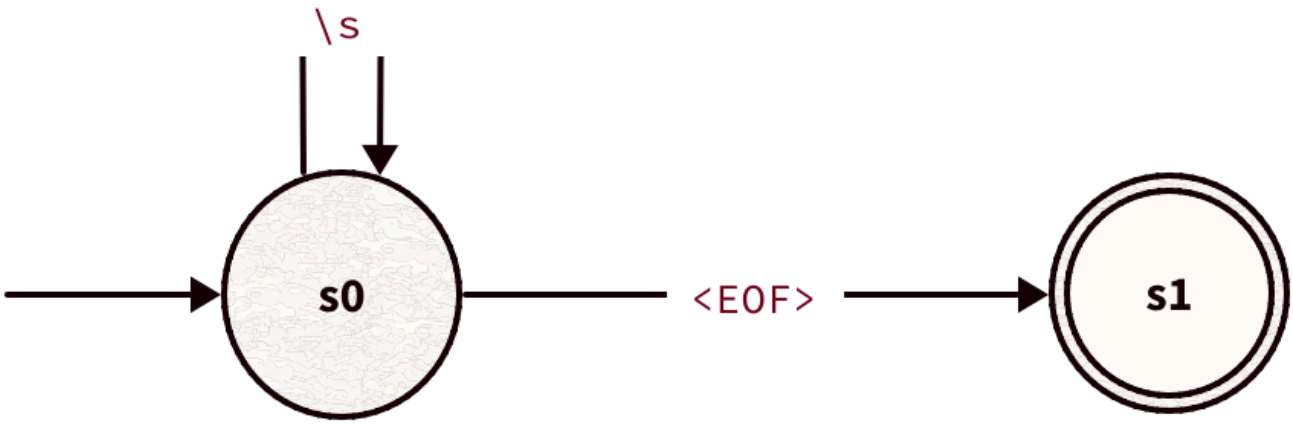
- `operator` :



- `whitespace` :



- `eof` :



Implementation Details

- **Keywords and identifiers**: It is easier to determine keywords as a final step after scanning for `identifier`s. If you accept an identifier *and* it is in a table of keywords, return a `keyword` token, otherwise return an `identifier` token.
- **Literal numbers**: It is easier to implement the literal numbers as a single DFA that will look for a literal `int` (and accept) or a literal `float` (if a decimal point is present). If the accept state is the one for `int`, then return a `literal-int` token; if it is `float`, return a `literal-float` token.
- **Literal strings**: We allow **escaped quotes** (`\"`) inside string literals. So, when you are looking for the closing quote of a string literal, you need to be aware of whether the preceding character was '`\`' or not. You can check this from the lexeme that you are building (no need for backtracking).
- **Literal `bool`**: The `true` and `false` literals for the `bool` type also match the pattern of identifiers, and are best handled as a post-check step like keywords (See "Keywords and identifiers" note above).
- **Newline continuation**: The `\` at the end of a line (for statement continuation) is absorbed by the scanner along with the newline that follows it, and the scanner emits a `LINE-CONTINUATION` token with a `null` lexeme. The parser treats this token like whitespace and ignores it; it is emitted so that the parser can still count source lines accurately (each `LINE-CONTINUATION` token represents exactly one consumed newline). In addition, it is "nice" to also absorb any trailing whitespace in case the line doesn't end immediately after the `\`. Technically, our syntax doesn't state that that is allowed, but it is a user-friendly "syntactic sugar" to add. **You have to be careful** not to respond to escape characters inside string literals as if they are newline continuations though. You can use precedence within your scanner to avoid this issue trivially. This enforces the rule that strings cannot be broken across lines.
- It is pragmatic to modularize the DFAs for each different kind of token, and create a function to perform the scanning for each. If you do this, it is helpful for your functions to be able to return something to indicate that there is no match at all to their respective pattern. This could be a special `NA` token, or a value like `None` (in Python) or whatever is similar in your implementation language. Example: Let's say that you are trying to scan for a closing brace `}` but you "see" a `5` character instead. This isn't an error, it just means that there is no work for the "find a closing brace" DFA to do… Return your `NA` representation so that the next DFA in line can get to work.
- **Whitespace**: We emit a `whitespace` token whenever we see one or more of *any* whitespace character contiguously. The lexeme holds the string of whitespace characters we consume, and the token class is `whitespace`. All contiguous whitespace characters should be consumed.
- **End-of-File (EOF)**: The `eof` token class is emitted whenever the end-of-file condition is encountered while reading from the input stream. There are two ways this can happen: At the end of a string of whitespace, or immediately following some non-whitespace token. If EOF is encountered while building a `whitespace` token, the lexeme is retained but the token class must be emitted as `eof`. If the EOF is encountered at the end of any other token type, the token should be emitted as usual, with the usual lexeme and token class, then an `eof` token with a null lexeme should be emitted next. (This is usually naturally handled by noticing the EOF condition when a reading loop has finished with consuming a word, and comes around to find the EOF flag set.) If the EOF condition occurs when

attempting to read the *next* character after finishing a token (which may happen depending on how you have written your token reading loop), then the `eof` token should be emitted with a null lexeme. Upon emitting an `eof` token, the action of the scanner is complete and no further tokens may be emitted.